

Section A

Q1. Which development methodology would you recommend for the fintech project, and why?

I would recommend the Agile methodology because the project has frequently changing requirements and the client wants to review working features every two weeks.

Agile divides the project into small iterations or sprints. At the end of each sprint, a working feature can be demonstrated to the client. Feedback can then be used in the next sprint.

If the team chooses the wrong methodology, such as Waterfall, two major risks are:

1. Difficulty handling changing requirements — changes later in development can become expensive and difficult.
2. Late feedback — the client may only see the completed product near the end, causing major rework if the product does not meet expectations.

Q2. What Git workflow should the team adopt?

The team should use a branch-based Git workflow instead of allowing everyone to directly modify the main branch.

A suitable workflow is:

1. Keep the main branch stable.
2. Create a separate branch for every new feature or change.
3. Make and test changes on the feature branch.
4. Commit the changes.
5. Push the branch to the remote repository.
6. Create a Pull Request/Merge Request.
7. Another teammate reviews the code.
8. Merge the branch into main only after approval and testing.

Important Git commands include:

```
git checkout -b feature-login
```

This creates and switches to a new feature branch.

```
git add .
```

```
git commit -m "Add login feature"
```

These commands stage and save the changes.

```
git push origin feature-login
```

This uploads the feature branch to the remote repository.

```
git pull origin main
```

This gets the latest changes from the main branch.

This workflow prevents incomplete code from being directly pushed to main.

Q3. What HTML input types and validation should be used?

The appropriate HTML input types are:

Field	HTML Input Type
-------	-----------------

Applicant Name	text
----------------	------

Email	email
-------	-------

Phone Number	tel
--------------	-----

Preferred Domain Select	
-------------------------	--

Resume	file
--------	------

Example:

```
<form>
```

```
<label>Name:</label>
```

```
<input type="text" name="name" required>
```

```
<label>Email:</label>
```

```
<input type="email" name="email" required>
```

```
<label>Phone:</label>
```

```
<input type="tel" name="phone" required>
```

```
<label>Preferred Domain:</label>
```

```
<select name="domain" required>
```

```
  <option value="">Select Domain</option>
```

```
  <option>Web</option>
```

```
  <option>Data</option>
```

```
  <option>Mobile</option>
```

```
  <option>AI</option>
```

```
</select>
```

```
<label>Resume:</label>
```

```
<input type="file" name="resume" required>
```

```
<input type="submit" value="Submit">
```

```
</form>
```

The required attribute ensures that the fields cannot be submitted empty.

The type="email" attribute provides built-in browser validation for the email format.

However, HTML5 validation alone is not sufficient for production-level validation because client-side validation can be bypassed. Server-side validation is also required for security and data integrity.

Q4. Explain Bootstrap's responsive grid system.

Bootstrap divides a row into 12 columns. Different Bootstrap classes can be used to specify how many columns an element occupies at different screen sizes.

For the news portal:

- Desktop → 3 columns, so each article occupies 4 Bootstrap columns because $12 / 3 = 4$.
- Mobile → 1 column, so each article occupies all 12 columns.

The class combination can be:

```
<div class="col-12 col-md-4">
```

Article

```
</div>
```

Explanation:

- col-12 → the article takes all 12 columns on small/mobile screens.
- col-md-4 → from medium-sized screens upward, the article takes 4 columns.
- Three col-md-4 elements therefore create a 3-column desktop layout.

Example:

```
<div class="row">
```

```
<div class="col-12 col-md-4">Article 1</div>
```

```
<div class="col-12 col-md-4">Article 2</div>
```

```
<div class="col-12 col-md-4">Article 3</div>
```

```
</div>
```

This produces a single-column mobile layout and a three-column larger-screen layout.

Q5. Why is an array suitable for rainfall data?

An array is suitable because we need to store 30 related rainfall readings. Instead of creating 30 separate variables, we can store all values in one array.

For example:

```
float rainfall[30];
```

The values can be accessed using indexes:

```
rainfall[0]
```

```
rainfall[1]
```

```
rainfall[2]
```

The program should use two logical steps:

Step 1 — Calculate average

Add all 30 rainfall values and divide the total by 30.

```
average = total / 30
```

Step 2 — Find above-average days

Traverse the array again and compare every reading with the calculated average.

```
if rainfall[i] > average
```

```
    print day
```

Two loops are needed because we cannot know the final average until all 30 values have been processed. Only after calculating the average can we correctly determine which days are above it.

Q6. Why does the pointer-based string program crash on empty input?

Answer:

When the user presses Enter without entering any characters, the string is empty. A C string still contains the terminating null character '\0'.

For example:

```
char name[50];
```

```
name[0] = '\0';
```

If a pointer is used to traverse the string, the program must check whether the current character is '\0' before processing or dereferencing further data.

Example:

```
char *ptr = name;
```

```
if (*ptr == '\0') {  
    printf("Empty string");  
}
```

A pointer-based traversal may look like:

```
while (*ptr != '\0') {  
    printf("%c", *ptr);  
    ptr++;  
}
```

Array-index traversal looks like:

```
for (int i = 0; name[i] != '\0'; i++) {  
    printf("%c", name[i]);  
}
```

Array indexing is generally easier for beginners to understand and debug because the index and string boundaries are more visible.

Section B

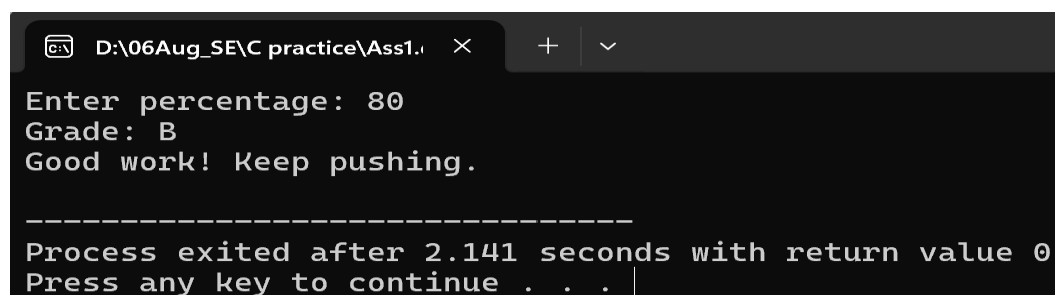
Q1. Grade Band Checker

The assessment requires the program to accept a percentage, assign A/B/C/D/F grades, print a motivational message, and reject values outside 0–100.

Answer:

```
1  #include <stdio.h>
2  int main(){
3      float percentage;
4      printf("Enter percentage: ");
5      scanf("%f", &percentage);
6      if (percentage < 0 || percentage > 100){
7          printf("Invalid percentage! Please enter a value between 0 and 100.\n");
8          return 0;
9      }
10
11     if (percentage >= 90)
12     {
13         printf("Grade: A\n");
14         printf("Excellent work! Keep it up.\n");
15     }
16     else if (percentage >= 75)
17     {
18         printf("Grade: B\n");
19         printf("Good work! Keep pushing.\n");
20     }
21     else if (percentage >= 60)
22     {
23         printf("Grade: C\n");
24         printf("Good effort! You can improve further.\n");
25     }
26     else if (percentage >= 45)
27     {
28         printf("Grade: D\n");
29         printf("Keep practicing and improve your score.\n");
30     }
31     else
32     {
33         printf("Grade: F\n");
34         printf("Don't give up! Work harder and try again.\n");
35     }
36
37     return 0;
38 }
```

Sample Output



```
D:\06Aug_SE\C practice\Ass1. >
Enter percentage: 80
Grade: B
Good work! Keep pushing.

-----
Process exited after 2.141 seconds with return value 0
Press any key to continue . . .
```

Q2. Weekly Study Hours Analyser

The program must store 7 daily values in an array, calculate total and average, find the highest-study day, and display a simple bar. Values outside 0–24 must be rejected and re-entered.

Answer:


```

1  #include <stdio.h>
2
3  int main()
4  {
5      float hours[7];
6      float total = 0;
7      float average;
8      int highestDay = 0;
9      int i, j;
10
11     for (i = 0; i < 7; i++)
12     {
13         do
14         {
15             printf("Enter study hours for Day %d: ", i + 1);
16             scanf("%f", &hours[i]);
17
18             if (hours[i] < 0 || hours[i] > 24)
19             {
20                 printf("Invalid input! Enter hours between 0 and 24.\n");
21             }
22
23             } while (hours[i] < 0 || hours[i] > 24);
24
25             total += hours[i];
26
27             if (hours[i] > hours[highestDay])
28             {
29                 highestDay = i;
30             }
31         }
32
33         average = total / 7;
34
35         printf("\n----- Weekly Report ----- \n");
36         printf("Total Study Hours: %.2f\n", total);
37         printf("Daily Average: %.2f\n", average);
38         printf("Highest Study: Day %d (%.2f hours)\n",
39             highestDay + 1, hours[highestDay]);
40
41         printf("\nStudy Chart:\n");
42
43         for (i = 0; i < 7; i++){
44             printf("Day %d: ", i + 1);
45             for (j = 0; j < (int)hours[i]; j++){
46                 printf("*");
47             }
48
49             printf("\n");
50         }
51         return 0;
52     }

```

Sample Output

```
D:\06Aug_SE\C practice\Ass2. X + v
Enter study hours for Day 1: 2
Enter study hours for Day 2: 3
Enter study hours for Day 3: 4
Enter study hours for Day 4: 5
Enter study hours for Day 5: 6
Enter study hours for Day 6: 7
Enter study hours for Day 7: 8

----- Weekly Report -----
Total Study Hours: 35.00
Daily Average: 5.00
Highest Study: Day 7 (8.00 hours)

Study Chart:
Day 1: **
Day 2: ***
Day 3: ****
Day 4: *****
Day 5: *****
Day 6: *****
Day 7: *****

-----
Process exited after 9.418 seconds with return value 0
Press any key to continue . . . |
```

Q3. Student Record Manager

The assessment requires a Student structure, an assignGrade() function, records for 3 students, and a printTopper() function.

```

1  #include <stdio.h>
2  struct Student{
3      char name[50];
4      int rollno;
5      float marks;
6      char grade;
7  };
8
9  void assignGrade(struct Student *s){
10     if (s->marks >= 90)
11         s->grade = 'A';
12     else if (s->marks >= 75)
13         s->grade = 'B';
14     else if (s->marks >= 60)
15         s->grade = 'C';
16     else if (s->marks >= 45)
17         s->grade = 'D';
18     else
19         s->grade = 'F';
20 }
21
22 void printTopper(struct Student students[], int n){
23     int i;
24     int topper = 0;
25     for (i = 1; i < n; i++)
26     {
27         if (students[i].marks > students[topper].marks)
28         {
29             topper = i;
30         }
31     }
32
33     printf("\nTopper: %s\n", students[topper].name);
34     printf("Marks: %.2f\n", students[topper].marks);
35 }
36
37 int main()
38 {
39     struct Student students[3];
40     int i;
41     printf("Enter details of 3 students:\n");
42
43     for (i = 0; i < 3; i++)
44     {
45         printf("\nStudent %d\n", i + 1);
46
47         printf("Enter name: ");
48         scanf("%[^\\n]", students[i].name);
49
50         printf("Enter roll number: ");
51         scanf("%d", &students[i].rollno);
52
53         printf("Enter marks: ");

```

```

54         scanf("%f", &students[i].marks);
55
56         assignGrade(&students[i]);
57     }
58
59     printf("\n-----\n");
60     printf("%-20s %-10s %-10s %-5s\n",
61         "Name", "Roll No", "Marks", "Grade");
62     printf("-----\n");
63
64     for (i = 0; i < 3; i++)
65     {
66         printf("%-20s %-10d %-10.2f %-5c\n",
67             students[i].name,
68             students[i].rollno,
69             students[i].marks,
70             students[i].grade);
71     }
72     printTopper(students, 3);
73     return 0;
74 }
}
28     {
29         topper = i;
30     }
31 }
32
33 printf("\nTopper: %s\n", students[topper].name);
34 printf("Marks: %.2f\n", students[topper].marks);
35 }
36
37 int main()
38 {
39     struct Student students[3];
40     int i;
41     printf("Enter details of 3 students:\n");
42
43     for (i = 0; i < 3; i++)
44     {
45         printf("\nStudent %d\n", i + 1);
46
47         printf("Enter name: ");
48         scanf("%[^\\n]", students[i].name);
49
50         printf("Enter roll number: ");
51         scanf("%d", &students[i].rollno);
52
53         printf("Enter marks: ");

```

Sample Output

```
D:\06Aug_SE\C practice\Ass3. X + v
Enter details of 3 students:

Student 1
Enter name: harita
Enter roll number: 73468
Enter marks: 67

Student 2
Enter name: fsduh
Enter roll number: 834957
Enter marks: 76

Student 3
Enter name: iluaesh
Enter roll number: 8378956
Enter marks: 72

-----
Name                Roll No    Marks    Grade
-----
harita              73468     67.00    C
fsduh               834957     76.00    B
iluaesh             8378956     72.00    C

Topper: fsduh
Marks: 76.00

-----
Process exited after 22.9 seconds with return value 0
Press any key to continue . . . |
```

Q4. Personal Expense Logger

The required program uses a structure, an array of up to 10 expenses, a menu, and expenses.txt for saving records.

```

1  #include <stdio.h>
2  struct Expense
3  {
4      char category[30];
5      float amount;
6  };
7  int main()
8  {
9      struct Expense expenses[10];
10     int count = 0;
11     int choice;
12     int i;
13     float total;
14     do
15     {
16         printf("\n===== Expense Logger =====\n");
17         printf("1. Add Expense\n");
18         printf("2. View All Expenses\n");
19         printf("3. Save & Exit\n");
20         printf("Enter your choice: ");
21         scanf("%d", &choice);
22
23         if (choice == 1)
24         {
25             if (count >= 10)
26             {
27                 printf("Expense limit reached!\n");
28
29             }
30             else
31             {
32                 printf("Enter category: ");
33                 scanf(" %[^\\n]", expenses[count].category);
34
35                 printf("Enter amount: ");
36                 scanf("%f", &expenses[count].amount);
37
38                 count++;
39
40                 printf("Expense added successfully!\n");
41             }
42         }
43         else if (choice == 2)
44         {
45             total = 0;
46
47             printf("\n----- All Expenses -----\\n");
48
49             if (count == 0)
50             {
51                 printf("No expenses recorded.\\n");
52             }
53             else

```

```

54 {
55     for (i = 0; i < count; i++)
56     {
57         printf("%d. %-20s %.2f\n",
58             i + 1,
59             expenses[i].category,
60             expenses[i].amount);
61
62         total += expenses[i].amount;
63     }
64
65     printf("-----\n");
66     printf("Running Total: %.2f\n", total);
67 }
68
69 else if (choice == 3)
70 {
71     FILE *fp;
72
73     fp = fopen("expenses.txt", "w");
74
75     if (fp == NULL)
76     {
77         printf("Error opening file!\n");
78         return 1;
79     }
80
81     for (i = 0; i < count; i++)
82     {
83         fprintf(fp, "%s,%.2f\n",
84             expenses[i].category,
85             expenses[i].amount);
86     }
87
88     fclose(fp);
89
90     printf("Expenses saved successfully.\n");
91     printf("Exiting program...\n");
92 }
93 else
94 {
95     printf("Invalid choice!\n");
96 }
97
98 } while (choice != 3);
99
100 return 0;
101 }

```

Section C

Q. Student Productivity Tracker

The project must combine arrays, structures, functions, menus, and file handling. It should store at least 3 subjects and 7 days of study hours and save the records to productivity_log.txt.


```

1  #include <stdio.h>
2  #define SUBJECTS 3
3  #define DAYS 7
4  struct StudyLog{
5      char subject[40];
6      float hours[DAYS];
7  };
8
9  void logHours(struct StudyLog study[]){
10     int subject, day;
11     printf("\n==== Log Today's Study Hours ==== \n");
12     printf("Select Day: \n");
13     for (day = 0; day < DAYS; day++){
14         printf("%d. Day %d \n", day + 1, day + 1);
15     }
16
17     printf("Enter day number: ");
18     scanf("%d", &day);
19
20     if (day < 1 || day > 7)
21     {
22         printf("Invalid day! \n");
23         return;
24     }
25
26     day--;
27
28     for (subject = 0; subject < SUBJECTS; subject++){
29         printf("Enter hours for %s: ",
30             study[subject].subject);
31
32         scanf("%f", &study[subject].hours[day]);
33
34         if (study[subject].hours[day] < 0)
35         {
36             printf("Invalid hours! Setting to 0. \n");
37             study[subject].hours[day] = 0;
38         }
39     }
40 }
41
42
43 void weeklyReport(struct StudyLog study[])
44 {
45     int i, j;
46     float total, average;
47
48     printf("\n===== WEEKLY REPORT ===== \n");
49
50     for (i = 0; i < SUBJECTS; i++)
51     {
52         total = 0;
53

```

54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78

```
for (j = 0; j < DAYS; j++)
{
    total += study[i].hours[j];
}

average = total / DAYS;

printf("\nSubject: %s\n", study[i].subject);
printf("Weekly Total: %.2f hours\n", total);
printf("Daily Average: %.2f hours\n", average);

printf("Progress Chart:\n");

for (j = 0; j < DAYS; j++)
{
    printf("Day %d: ", j + 1);

    int k;

    for (k = 0; k < (int)study[i].hours[j]; k++)
    {
        printf(".");
    }

    printf("\n");
}
```

```

80     }
81 }
82
83 void saveData(struct StudyLog study[])
84 {
85     FILE *fp;
86     int i, j;
87
88     fp = fopen("productivity_log.txt", "w");
89
90     if (fp == NULL)
91     {
92         printf("Error creating file!\n");
93         return;
94     }
95
96     for (i = 0; i < SUBJECTS; i++)
97     {
98         fprintf(fp, "%s", study[i].subject);
99
100         for (j = 0; j < DAYS; j++)
101         {
102             fprintf(fp, ",%.2f", study[i].hours[j]);
103         }
104
105         fprintf(fp, "\n");
106     }
107
108     fclose(fp);
109
110     printf("Data saved successfully to productivity_log.txt\n");
111 }
112
113 int main()
114 {
115     struct StudyLog study[SUBJECTS] =
116     {
117         {"C Programming", {0, 0, 0, 0, 0, 0, 0}},
118         {"Python", {0, 0, 0, 0, 0, 0, 0}},
119         {"Web Development", {0, 0, 0, 0, 0, 0, 0}}
120     };
121
122     int choice;
123
124     do
125     {
126         printf("\n==== STUDENT PRODUCTIVITY TRACKER =====\n");
127         printf("1. Log Today's Study Hours\n");

```

```

128     printf("2. View Weekly Report\n");
129     printf("3. Save & Exit\n");
130
131     printf("Enter your choice: ");
132     scanf("%d", &choice);
133
134     switch (choice)
135     {
136         case 1:
137             logHours(study);
138             break;
139
140         case 2:
141             weeklyReport(study);
142             break;
143
144         case 3:
145             saveData(study);
146             printf("Program ended.\n");
147             break;
148
149         default:
150             printf("Invalid choice! Try again.\n");
151     }
152
153     } while (choice != 3);
154
155     return 0;
156 }

```

Output:

1. Log Today's Study Hours
2. View Weekly Report
3. Save & Exit

Enter your choice: 1

==== Log Today's Study Hours ====

Select Day:

1. Day 1
2. Day 2
3. Day 3
4. Day 4
5. Day 5
6. Day 6
7. Day 7

Enter day number: 2

Enter hours for C Programming: 3

Enter hours for Python: 6

Enter hours for Web Development: 4

==== STUDENT PRODUCTIVITY TRACKER ====

1. Log Today's Study Hours
2. View Weekly Report
3. Save & Exit

Enter your choice: 3

Data saved successfully to productivity_log.txt

Program ended.

Process exited after 13.09 seconds with return value 0

Press any key to continue . . . |

Section D — AI-Augmented Learning

Q. Write a C program that accepts 10 integers and performs the required operations.

The assessment requires 10 integers, maximum, minimum, mean, ascending sorting, and determining whether the mean is closer to the minimum, maximum, or exactly midway.

```
1  #include <stdio.h>
2  int main()
3  {
4      int arr[10];
5      int i, j;
6      int max, min;
7      int temp;
8      float sum = 0;
9      float mean;
10     float mid;
11
12     printf("Enter 10 integers:\n");
13
14     for (i = 0; i < 10; i++)
15     {
16         printf("Enter number %d: ", i + 1);
17         scanf("%d", &arr[i]);
18
19         sum += arr[i];
20     }
21
22     min = arr[0];
23     max = arr[0];
24
25     for (i = 1; i < 10; i++)
26     {
27         if (arr[i] < min)
```

```

28         min = arr[i];
29
30         if (arr[i] > max)
31             max = arr[i];
32     }
33
34     mean = sum / 10;
35
36     printf("\nMaximum = %d\n", max);
37     printf("Minimum = %d\n", min);
38     printf("Mean = %.2f\n", mean);
39
40     /* Bubble Sort */
41     for (i = 0; i < 9; i++)
42     {
43         for (j = 0; j < 9 - i; j++)
44         {
45             if (arr[j] > arr[j + 1])
46             {
47                 temp = arr[j];
48                 arr[j] = arr[j + 1];
49                 arr[j + 1] = temp;
50             }
51         }
52     }
53
54     printf("\nSorted Array:\n");
55
56     for (i = 0; i < 10; i++)
57     {
58         printf("%d ", arr[i]);
59     }
60
61     printf("\n");
62
63     mid = (min + max) / 2.0;
64
65     if (mean == mid)
66     {
67         printf("Mean is exactly midway between minimum and maximum.\n");
68     }
69     else if (mean < mid)
70     {
71         printf("Mean is closer to the minimum.\n");
72     }
73     else
74     {
75         printf("Mean is closer to the maximum.\n");
76     }
77
78     return 0;
79 }

```


Example

For:

10 20 30 40 50 60 70 80 90 100

Output:

```
D:\06Aug_SE\C practice\Ass5.  X  +  v
Enter 10 integers:
Enter number 1: 34
Enter number 2: 6
Enter number 3: 4
Enter number 4: 25
Enter number 5: 86
Enter number 6: 35
Enter number 7: 87
Enter number 8: 36
Enter number 9: 08
Enter number 10: 54

Maximum = 87
Minimum = 4
Mean = 37.50

Sorted Array:
4 6 8 25 34 35 36 54 86 87
Mean is closer to the minimum.

-----
Process exited after 11.62 seconds with return value 0
Press any key to continue . . . |
```